

Informe del trabajo práctico

Ecualización local de histograma y validación
automática de planillas de calificaciones

CARRERA	Tecnicatura Universitaria en Inteligencia Artificial Facultad de Ciencias Exactas, Ingeniería y Agrimensura - UNR
MATERIA	Procesamiento de Imágenes I (IA 4.4)
TRABAJO	Trabajo Práctico N° 1 - Problemas 1 y 2
INTEGRANTES	Nino Busalacchi Lucas Casariego Ticiano Licarzze Emanuel Bustos

PROBLEMA 1

Ecualización local de histograma

1.1 Qué nos piden

Tenemos una imagen en escala de grises (Figura 1) con cinco cuadrados negros sobre un fondo gris claro. Dentro de cada cuadrado hay un objeto escondido, con un tono tan parecido al negro que no se ve. Nos piden programar una función de **ecualización local de histograma** que reciba la imagen y el tamaño de la ventana ($M \times N$), usarla para descubrir los objetos ocultos y analizar cómo influye el tamaño de la ventana.

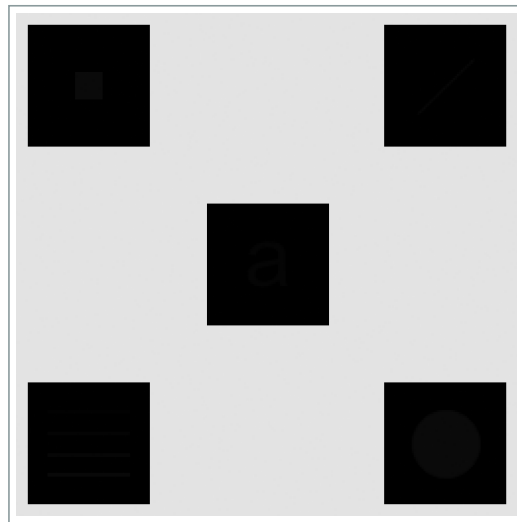


Figura 1. Imagen de entrada (256 x 256 píxeles). A simple vista los cuadrados parecen negros.

1.2 Ideas previas

Histograma: es una tabla que cuenta cuántos píxeles hay de cada tono de gris, del 0 (negro) al 255 (blanco).

Ecualización: es una forma de mejorar el contraste "estirando" los tonos para que usen todo el rango de 0 a 255. Se calcula con la **CDF** (distribución acumulada), que para cada tono dice qué parte de los píxeles es igual o más oscura que él. El nuevo valor es $255 \cdot \text{CDF}(\text{tono})$.

Global o local: la ecualización global usa un único histograma de toda la imagen, así que los objetos escondidos, que son pocos píxeles, siguen viéndose negros. La local, en cambio, usa sólo los píxeles vecinos: dentro de un cuadrado negro, un píxel apenas más claro que sus vecinos pasa a verse blanco.

La ventana: es un rectángulo de $M \times N$ píxeles que se centra en cada píxel. Se calcula el histograma de la ventana y el nuevo valor se aplica sólo al píxel del centro; después la ventana se corre un lugar. Usamos Python con NumPy, OpenCV y Matplotlib.

1.3 Cómo lo resolvimos

Paso 1 - Abrir la imagen y validar los datos

Leemos la imagen en escala de grises y, si no se encuentra, mostramos un error. La función `ecualizacion_local` revisa primero que la imagen sea en escala de grises y de tipo `uint8` (valores de 0 a 255), y que M y N sean enteros positivos.

Paso 2 - Agregar un borde

En los bordes de la imagen la ventana se "sale". Por eso agregamos un marco con `cv2.copyMakeBorder`, copiando los píxeles del borde. Calculamos cuánto agregar de cada lado a partir de `M` y `N`, así también funcionan las ventanas rectangulares y las pares (en ese caso la ventana queda con un vecino más hacia arriba y hacia la izquierda).

problema1.py, líneas 34-37

```
L = 256
top, left = M // 2, N // 2
bottom, right = M - 1 - top, N - 1 - left
img_pad = cv2.copyMakeBorder(img, top, bottom, left, right, cv2.BORDER_REPLICATE)
```

Paso 3 - Recorrer la imagen

Recorremos la imagen píxel por píxel. En cada posición tomamos la ventana, calculamos su histograma con `np.histogram`, su CDF con `cumsum` y le asignamos al píxel central el valor $255 \cdot \text{CDF}$.

problema1.py, líneas 41-47

```
for i in range(img.shape[0]):
    for j in range(img.shape[1]):
        ventana = img_pad[i:i + M, j:j + N]
        # Histograma de la ventana y distribución acumulada.
        hist, bins = np.histogram(ventana.flatten(), L, [0, L])
        cdf = hist.cumsum() / area
        salida[i, j] = np.round((L - 1) * cdf[img[i, j]])
```

Paso 4 - Comparar

En el main comparamos la ecualización global (`cv2.equalizeHist`) con la local de 15×15 , y después probamos ventanas de 3×3 , 7×7 , 15×15 , 31×31 , 51×51 , 5×25 y 25×5 . Cada comparación se guarda como imagen.

1.4 Problemas que tuvimos

En los resultados el fondo aparece lleno de puntos negros. Al revisar los valores de la imagen vimos que el fondo no es parejo: casi todo vale 227, pero hay unos 1.200 píxeles sueltos con 226 (y otros tantos con 228). A simple vista no se nota, pero la ecualización exagera esa diferencia: un píxel de 226 rodeado de 227 es "el más oscuro de su zona" y pasa a verse negro. Esto ocurre tanto en la ecualización global como en la local, así que no es un error del código.

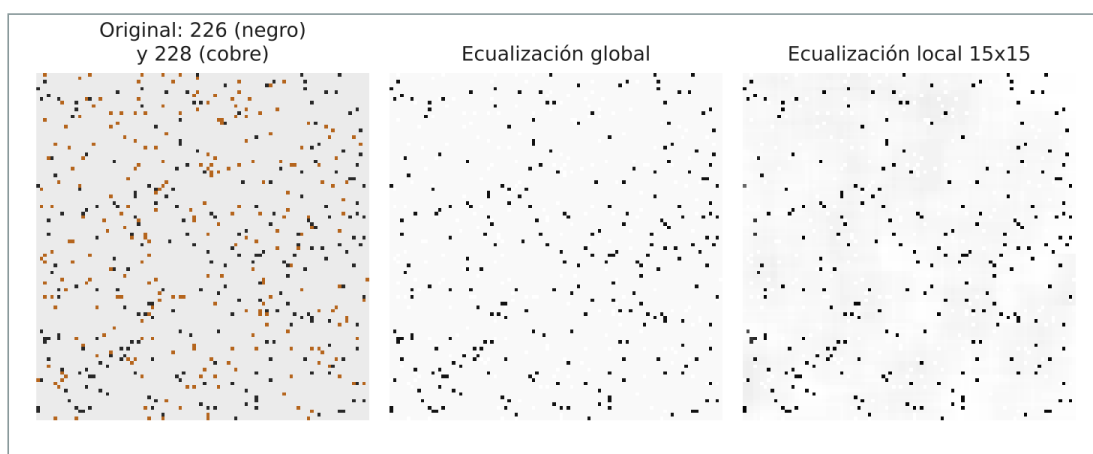


Figura 2. Un recorte del fondo. Izquierda: los píxeles con 226 (negro) y 228 (cobre). Al ecualizar, los puntos negros aparecen justo donde estaban los 226. Imagen generada al correr el código.

Además, como se calcula un histograma por cada píxel, el programa tarda: en nuestra prueba, correr todas las comparaciones llevó unos 27 segundos.

1.5 Resultados

En la Figura 3, la ecualización global deja el fondo blanco pero los cuadrados siguen negros y los objetos no se ven. Con la local de 15 x 15 aparecen claramente los cinco objetos.

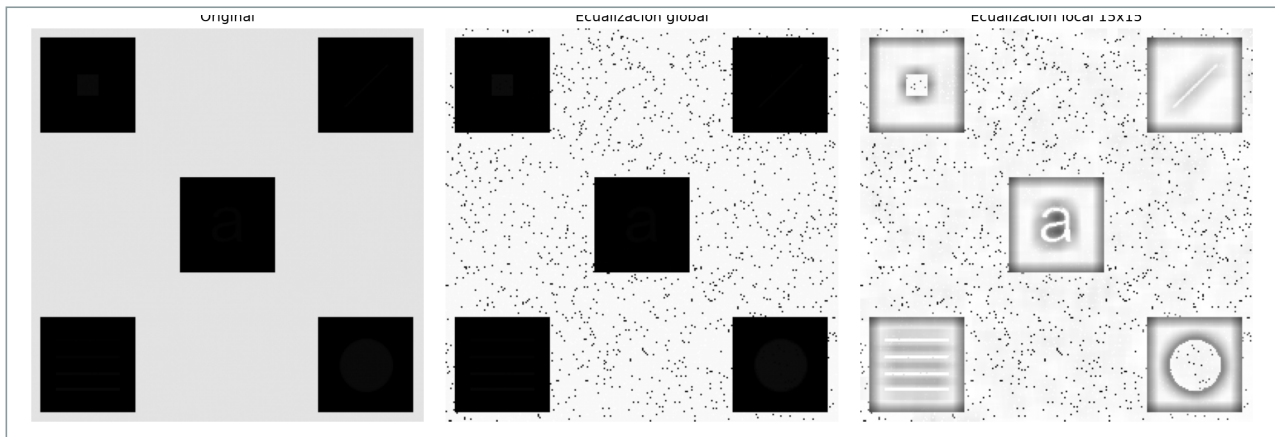


Figura 3. Original, ecualización global y ecualización local con ventana de 15 x 15.

En la Figura 4 se ve el efecto del tamaño de la ventana:

- **3 x 3:** sólo se ven los contornos de los objetos, y el fondo se llena de ruido.
- **7 x 7 y 15 x 15:** los objetos se ven completos y nítidos; 15 x 15 es el resultado más claro.
- **31 x 31 y 51 x 51:** los objetos siguen visibles, pero los cuadrados se oscurecen hacia los bordes y los detalles finos (la línea diagonal y las líneas horizontales) pierden contraste.
- **5 x 25 y 25 x 5:** las ventanas rectangulares "estiran" el efecto: aparece una sombra hacia los costados en la primera y hacia arriba y abajo en la segunda.

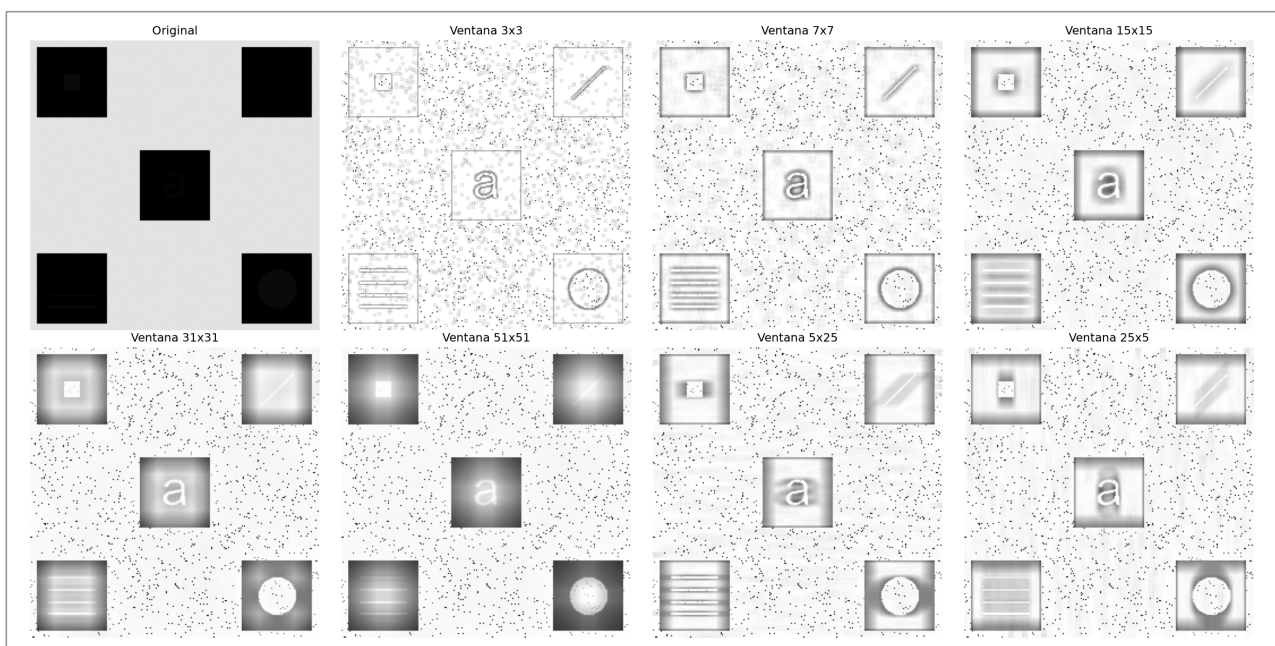


Figura 4. Resultados de la ecualización local con distintos tamaños de ventana.

1.6 Conclusiones

Los objetos ocultos son: un **cuadrado pequeño** (arriba a la izquierda), una **línea diagonal** (arriba a la derecha), la **letra "a"** (centro), **cuatro líneas horizontales** (abajo a la izquierda) y un **círculo** (abajo a la derecha).

La ecualización local muestra detalles que la global no puede, porque ajusta el contraste en cada zona. Como desventajas, exagera el ruido, es lenta y el resultado depende del tamaño de la ventana: muy chica sólo marca bordes, muy grande se parece a la global. Para esta imagen, una ventana mediana (alrededor de 15 x 15) fue la mejor opción.

PROBLEMA 2

Validación de planillas de calificaciones

2.1 Qué nos piden

Tenemos cuatro planillas de notas escaneadas como imagen (Figura 5). Cada fila es un alumno con seis campos, y hay que controlar automáticamente que cada uno cumpla estas reglas:

Campo	Regla
Legajo	8 caracteres, una sola palabra
Nombre y Apellido	al menos 2 palabras y no más de 12 caracteres
Parcial 1, 2 y 3	1 o 2 caracteres, sin espacios
Condición Final	un solo carácter

El programa debe mostrar por consola OK o MAL en cada campo, guardar los resultados en un CSV, generar una imagen con los nombres de los alumnos no aprobados (condición L o R) que estén bien cargados, y repetir todo para las cuatro planillas.



Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						

Figura 5. Esquema de la planilla vacía: siete columnas (Nro. y los seis campos) y 20 filas.

2.2 Ideas previas

Umbralado: pasar la imagen a blanco y negro puro; tomamos como "tinta" todo píxel con valor menor a 128.

Suma de filas y columnas: si sumamos la tinta de cada fila, las líneas de la tabla dan valores muy altos porque cruzan toda la hoja. Así encontramos dónde están las líneas.

Componentes conectadas: son grupos de píxeles de tinta que se tocan. En una celda, cada letra o número es un grupo, así que contarlos es contar caracteres.

Huecos de las letras: zonas blancas encerradas por tinta. La R tiene un hueco y la L ninguno; esto nos sirve para distinguirlas.

2.3 Cómo lo resolvimos

Detectar la grilla

En `detectar_grilla` sumamos la tinta por fila y nos quedamos con las que superan el 70 % del máximo: esas son las líneas horizontales. Hacemos lo mismo con las columnas para las verticales. En las cuatro planillas encontramos 22 líneas horizontales y 8 verticales.

Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1	C-1557/1	JUAN CARLINI	6	6	7	A
2	M-4471/7	BIANCA MONTE	1	4	4	L
3						
4	R-8755/6	RIVAS HUGO	8	2	6	R
5	F-5455/1	FARIAS JORGE	9	3	4	L
6	G-3314/6	JUANA GOMEZ	2	1	1	L
7	V-3122/8	NICOLAS VEGA	7	8	9	A
8	S-1867/6	AMANDA SANTOS	1	8	3	L
9	J-4789/4	LUCAS JARDON	8	8	7	A
10	F-1948/2	NAHUEL FAUSTO	9	3	1	L
11						
12	D-2123/6	JORGE DELGADO	9	6	2	R
13	S-1258/7	JOSE SANTINI	1	9	7	R
14						
15						
16	F-2319/9	JULIO FOGLIA	1	8	1	L
17	G-3197/2	CAMILA GODOY	9	5	9	R
18	G-2318/2	RENZO GARCIA	10	10	10	A
19	S-9472/2	MIGUEL SASTRE	6	6	7	A
20						

Figura 6. Líneas encontradas por `detectar_grilla` en la planilla 1: horizontales en cobre y verticales en verde. Imagen generada al correr el código.

```
tinta = img < 128
filas = tinta.sum(axis=1)
horizontales = intervalos(filas > 0.7 * filas.max())
if len(horizontales) < 3:
    raise ValueError("No se detectaron el encabezado y los registros de la grilla.")
# Medir columnas sólo en los registros evita que el logo o las celdas
# fusionadas del encabezado alteren la detección en planillas cortas.
columnas = tinta[horizontales[1][1]:horizontales[-1][0]].sum(axis=0)
verticales = intervalos(columnas > 0.7 * columnas.max())
```

problema2.py, líneas 28-36

Recortar las celdas y separar los caracteres

Cada celda se recorta entre dos líneas, sin incluirlas. Dentro de la celda buscamos las componentes conectadas y las ordenamos de izquierda a derecha. Si dos componentes están una encima de la otra (como la tilde y la N de la Ñ) las unimos en un solo carácter.

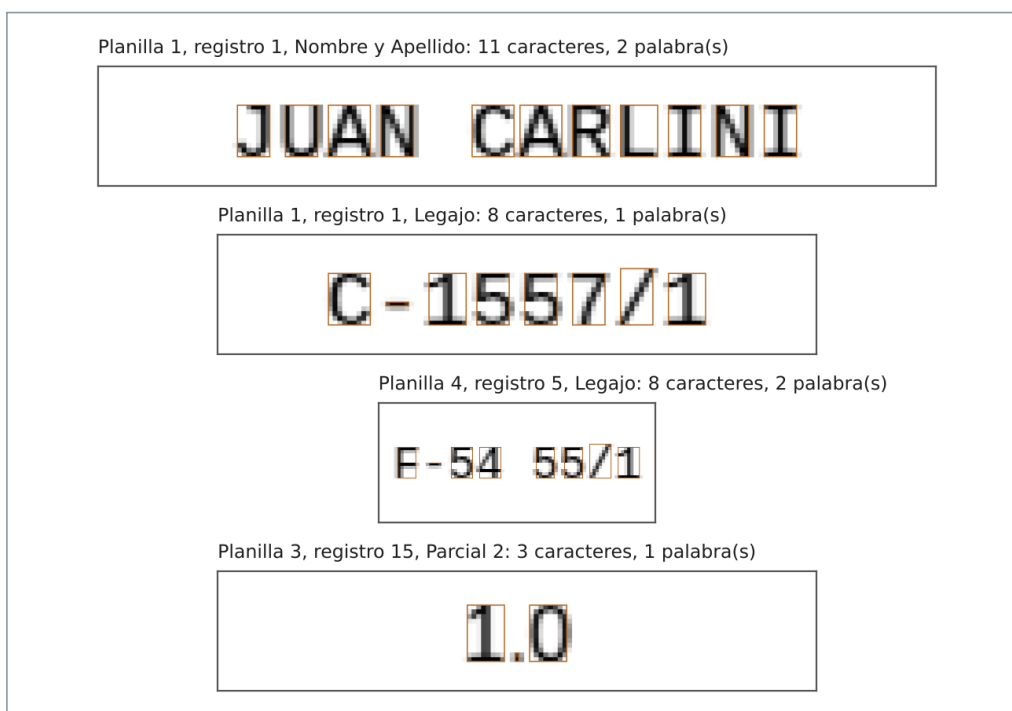


Figura 7. Caracteres encontrados (recuadros en cobre) en cuatro celdas, con la cantidad de caracteres y de palabras que cuenta el programa. Imagen generada al correr el código.

Contar palabras

Medimos el espacio entre cada carácter y el siguiente: si es mayor que la mitad de la altura de una letra, hay un espacio y empieza otra palabra.

Validar los campos

Con la cantidad de caracteres y de palabras de cada campo, `validar_campos` aplica las reglas de la tabla anterior:

```
def validar_campos(conteos):
    """Reglas de formato; los espacios separan palabras y no cuentan como letras."""
    (cl, pl), (cn, pn), *resto = conteos
    notas, (cc, _) = resto[:3], resto[3]
    validos = [cl == 8 and pl == 1, 0 < cn <= 12 and pn >= 2]
    validos.extend(c in (1, 2) and p == 1 for c, p in notas)
    validos.append(cc == 1)
    return ["OK" if valido else "MAL" for valido in validos]
```

problema2.py, líneas 76-83

Reconocer L y R

Si la condición tiene un solo carácter, `identificar_lr` mira su forma: la L tiene un trazo vertical, ningún hueco y una base ancha; la R tiene un hueco arriba y dos "patas" abajo. Esto sólo decide qué alumnos aparecen en la imagen de salida.

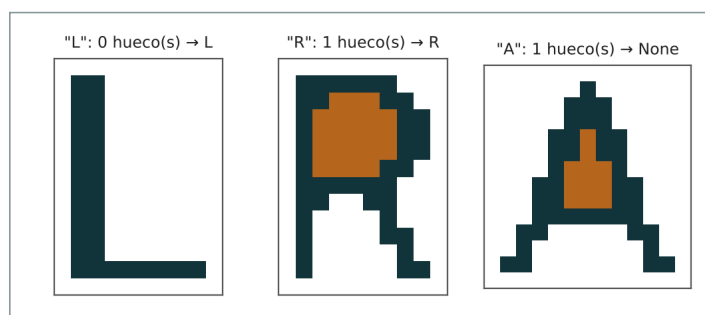


Figura 8. Huecos (en cobre) de tres letras de la columna Condición Final. La A tiene un hueco pero no dos patas abajo, así que no se toma como R. Imagen generada al correr el código.

Salidas

Para cada planilla se imprime el resultado por consola, se guarda el CSV y se genera la imagen de no aprobados, con el recorte del nombre y la etiqueta "LIBRE (L)" o "RECUPERA (R)". El main repite esto para las cuatro planillas.

2.4 Decisiones de diseño

- **No borramos componentes chicas.** El enunciado sugiere eliminarlas para sacar restos de las líneas, pero un guion o un punto también son chicos (por ejemplo en "1.0"). Como el recorte ya deja afuera las líneas, no hizo falta.
- **El espacio no cuenta en el límite de 12 caracteres.** El enunciado da la lista de caracteres permitidos y el espacio no está en ella, así que lo tomamos sólo como separador. Además, si lo contáramos, nombres bien cargados como "NAHUEL FAUSTO" o "AMANDA SANTOS" (12 letras + 1 espacio) quedarían MAL.
- **Medidas relativas.** Las planillas tienen distinto tamaño, por eso los umbrales se calculan a partir de cada imagen (70 % del máximo, mitad de la altura de letra) y no con números fijos.

2.5 Resultados

Por consola se ve, para cada registro, el resultado de cada campo. Por ejemplo:

```
Planilla: grade_sheet_2.png
Registro 1:
  Legajo: OK
  Nombre y Apellido: MAL
  Parcial 1: OK
  Parcial 2: OK
  Parcial 3: OK
  Condición Final: OK
[...]
Resumen: 20 registros, 3 completos OK, 3 no aprobados válidos.
```

salida de problema2.py (fragmento)

Resumen de las cuatro planillas:

Planilla	Registros	Todos los campos OK	No aprobados válidos
grade_sheet_1	20	15	10 (6 L, 4 R)
grade_sheet_2	20	3	3 (3 L, 0 R)
grade_sheet_3	20	0	0 (0 L, 0 R)
grade_sheet_4	20	6	4 (2 L, 2 R)

Algunos ejemplos de errores detectados:

Planilla / registro	Campo	Contenido	Motivo
2 / 4	Legajo	R-87455/6	9 caracteres
4 / 5	Legajo	F-54 55/1	tiene un espacio (2 palabras)
2 / 13	Nombre y Apellido	JOSE LUCAS SANTINI	16 caracteres
3 / 8	Nombre y Apellido	AMANDASANTOS	una sola palabra
2 / 16	Parcial 1	AUS	3 caracteres
3 / 15	Parcial 2	1.0	3 caracteres
2 / 10	Condición Final	LL	2 caracteres

Cada planilla genera además su CSV; por ejemplo, las primeras filas de `validacion_grade_sheet_2.csv` son:

ID	Legajo	Nombre y Apellido	Parcial 1	Parcial 2	Parcial 3	Condición Final
1	OK	MAL	OK	OK	OK	OK
2	OK	OK	OK	OK	OK	OK
3	MAL	MAL	OK	OK	OK	OK
4	MAL	OK	OK	OK	OK	OK
5	OK	OK	OK	OK	OK	OK

Las imágenes de no aprobados de cada planilla son las siguientes (en la planilla 3 ningún registro tiene todos sus campos OK, por eso sólo aparece el mensaje):

No aprobados - solo registros completos OK		
grade_sheet_1 / 2	BIANCA MONTE	LIBRE (L)
grade_sheet_1 / 4	RIVAS HUGO	RECUPERA (R)
grade_sheet_1 / 5	FARIAS JORGE	LIBRE (L)
grade_sheet_1 / 6	JUANA GOMEZ	LIBRE (L)
grade_sheet_1 / 8	AMANDA SANTOS	LIBRE (L)
grade_sheet_1 / 10	NAHUEL FAUSTO	LIBRE (L)
grade_sheet_1 / 12	JORGE DELGADO	RECUPERA (R)
grade_sheet_1 / 13	JOSE SANTINI	RECUPERA (R)
grade_sheet_1 / 16	JULIO FOGLIA	LIBRE (L)
grade_sheet_1 / 17	CAMILA GODOY	RECUPERA (R)

Figura 9. No aprobados de la planilla 1.

No aprobados - solo registros completos OK		
grade_sheet_2 / 2	BIANCA MONTE	LIBRE (L)
grade_sheet_2 / 5	FARIAS JORGE	LIBRE (L)
grade_sheet_2 / 8	AMANDA SANTOS	LIBRE (L)

Figura 10. No aprobados de la planilla 2.

No aprobados - solo registros completos OK

No hay registros validos con condicion L o R.

Figura 11. No aprobados de la planilla 3.

No aprobados - solo registros completos OK

grade_sheet_4 / 7 NICOLAS VEGA LIBRE (L)

grade_sheet_4 / 12 JORGE DELSIO RECUPERA (R)

grade_sheet_4 / 17 CAMILA GODOY RECUPERA (R)

grade_sheet_4 / 19 MIGUEL SASTRE LIBRE (L)

Figura 12. No aprobados de la planilla 4.

2.6 Conclusiones y limitaciones

El programa valida las cuatro planillas sin ajustes manuales, aunque tengan tamaños distintos, usando sólo técnicas vistas en la materia: umbralado, suma de filas y columnas y componentes conectadas.

Sus limitaciones: supone que la planilla tiene siete columnas; si dos letras se tocan se contarían como una; el reconocimiento de L y R está pensado para letra de imprenta; y, como pide el enunciado, sólo controla el formato, no si los datos son verdaderos.